Defensa de la máquina virtual simple para tres integrantes

Propósito de la exposición

Demostrar cómo una instrucción almacenada en memoria cambia el estado de una máquina virtual. La explicación debe conectar cada operación con PC, IR, ACC, memoria y el ciclo Fetch, Decode y Execute. La defensa completa dura aproximadamente 18 minutos, incluidos tres minutos de demostración.

Reemplazar Integrante 1, Integrante 2 e Integrante 3 por los nombres reales. Los tres deben conocer el ejemplo de suma y la diferencia entre un error de carga y un error de ejecución.

Reparto y tiempo

Integrante	Diapositivas	Tiempo previsto	Responsabilidad
Integrante 1	1 a 5	0:00 a 5:30	Problema, objetivo, arquitectura, memoria y registros.
Integrante 2	6 a 11	5:30 a 12:00	Instrucciones, ciclo, suma, demostración, saltos y dispositivos.
Integrante 3	12 a 16	12:00 a 18:00	Errores, reportes, pruebas, organización y conclusiones.

Una sola persona maneja el teclado durante toda la exposición: se recomienda el Integrante 2. Los demás señalan la pantalla y explican, evitando cambios de ventana simultáneos.

Preparación antes de entrar al aula

1. Copiar la carpeta completa del proyecto a la computadora de exposición. La aplicación usa Python 3.10 o superior con Tkinter. No requiere Internet ni paquetes externos para ejecutarse.
2. Abrir una terminal dentro de la carpeta del proyecto. Comprobar la instalación con `python --version` y `python -m tkinter`.
3. Ejecutar `python -m tests.verificar_entrega` y `python -m unittest discover -s tests -v`. Para comprobar controles reales de la interfaz, ejecutar `python -m tests.verificar_gui`.
4. Abrir la aplicación con `python simulador_vm.py` y dejar seleccionado el caso Suma. Ajustar el tamaño de la ventana para que todos los controles se vean en el proyector.
5. Abrir el PowerPoint y probar el modo presentación. Verificar que las imágenes estén embebidas y que los nombres de integrantes estén completos.
6. Dejar disponibles los reportes HTML y JSON de ejemplo. Tener una copia del PDF de la entrega, los programas ASM y el PowerPoint en una memoria USB.
7. Ensayar con cronómetro. Si exceden veinte minutos, reducir la lectura de instrucciones y conservar la explicación de la suma, los errores y las evidencias.

Guion del Integrante 1

Diapositiva 1 Máquina virtual simple

Tiempo aproximado: 35 segundos.

“Buenos días. Somos el grupo 13 y presentamos una máquina virtual simple para la asignatura Arquitectura de Sistemas Operativos. El programa representa un procesador didáctico con memoria y registros. Nuestro propósito es mostrar cómo una instrucción se obtiene desde memoria, se interpreta y produce un resultado. Durante la exposición veremos su arquitectura, ejecutaremos un programa y revisaremos las pruebas y los reportes que permiten comprobar su funcionamiento.”

Diapositiva 2 Problema y objetivo

Tiempo aproximado: 1 minuto.

“En una computadora real las instrucciones se ejecutan demasiado rápido para observarlas una por una. Normalmente vemos el resultado final, pero no los cambios intermedios de los registros. El problema educativo consiste en hacer visibles esas operaciones. Nuestra solución permite cargar un programa pequeño, avanzar por instrucciones completas y revisar el estado después de cada paso. El objetivo no es reemplazar una computadora real ni ejecutar sus programas; es comprender la relación entre memoria, unidad de control y registros.”

“La propuesta pide memoria simulada, PC, IR, ACC, ciclo de instrucción, carga de programas, ejecución por pasos y automática, errores y evidencias. Sobre esa base añadimos explicaciones y reportes de ejecución.”

Diapositiva 3 Arquitectura del simulador

Tiempo aproximado: 1 minuto y 15 segundos.

“La interfaz gráfica y la consola son dos formas de utilizar el mismo motor. La memoria guarda instrucciones y datos. La unidad de control coordina Fetch, Decode y Execute. Los registros representan el estado de la CPU. Los controladores adaptan el teclado, los archivos y la impresora virtual. Esta separación evita tener dos implementaciones distintas del procesador: un programa debe producir el mismo resultado tanto en consola como en la interfaz.”

Señalar en la tabla las responsabilidades de la unidad de control, memoria y registros. Explicar que el controlador de disco lee un archivo ASM y que la impresora es una salida simulada, no una impresora física.

Diapositiva 4 Memoria e instrucciones

Tiempo aproximado: 1 minuto y 15 segundos.

“La memoria tiene cien celdas, con direcciones de cero a noventa y nueve. Una celda contiene una instrucción o un número entero. El código se carga de manera contigua desde la dirección cero. Los datos pueden declararse con DATA en direcciones que no se superpongan con el código inicial. Los comentarios, las líneas vacías y las declaraciones DATA no consumen direcciones de instrucciones.”

“En el ejemplo de suma, las cinco instrucciones están en las direcciones cero a cuatro. En las celdas diez y once guardamos diez y veinte. La celda veinte recibirá el resultado treinta. Una dirección es la ubicación de una celda; no es necesariamente el número almacenado en ella.”

Diapositiva 5 Registros de la máquina

Tiempo aproximado: 1 minuto y 25 segundos.

“PC es el contador de programa: indica la dirección de la próxima instrucción. IR conserva la instrucción obtenida desde memoria. ACC es el acumulador, donde se reciben valores y se realizan sumas o restas. Implementamos además las banderas Z y N. Z vale uno si ACC es cero y N vale uno si ACC es negativo. Al iniciar, ACC es cero, por lo tanto Z inicia activa.”

“PC, IR y ACC tienen funciones diferentes. PC es una dirección; IR es una instrucción; ACC es un dato con el que se trabaja. Un salto modifica el PC, una carga modifica ACC y Fetch actualiza IR.”

Transición: “Con esta arquitectura definida, mi compañero mostrará las instrucciones y cómo cambia la máquina al ejecutarlas.”

Guion del Integrante 2

Diapositiva 6 Lenguaje de instrucciones

Tiempo aproximado: 55 segundos.

“El lenguaje tiene diez instrucciones. LOAD y STORE transfieren datos entre memoria y acumulador. ADD y SUB realizan aritmética. JMP cambia el flujo siempre; JZ y JNZ dependen de la bandera cero. INPUT recibe un entero, OUTPUT lo muestra y HALT termina. ACC es implícito: LOAD diez significa cargar el contenido de la celda diez, no cargar el número diez como valor inmediato.”

“DATA es una directiva del cargador. No es una operación de CPU y no consume ciclos. Los archivos son texto UTF-8 con una operación por línea.”

Diapositiva 7 Ciclo Fetch Decode Execute

Tiempo aproximado: 1 minuto.

“En Fetch se lee Memoria de PC y la instrucción se copia a IR. En Decode se reconoce el opcode y su operando. Después se prepara la continuación secuencial incrementando PC. En Execute se aplica la operación: puede cambiar ACC, memoria, salida o el destino de PC. Finalmente se actualizan las banderas y se registra el paso.”

“Un paso de la aplicación ejecuta una instrucción completa. Las tarjetas describen las tres fases del último paso; no representan tres clics separados. En un salto, Execute puede reemplazar el PC que ya se había incrementado.”

Diapositiva 8 Traza de la suma

Tiempo aproximado: 50 segundos.

“Este programa permite comprobar todos los cambios. LOAD diez copia diez al acumulador. ADD once suma veinte y deja treinta. STORE veinte copia treinta a memoria sin cambiar ACC. OUTPUT emite treinta y HALT termina. El resultado es ACC igual a treinta, memoria veinte igual a treinta, salida treinta y PC igual a cinco. HALT también cuenta como una instrucción completada.”

Señalar la tabla y evitar leer cada celda. Destacar el cambio producido por STORE y la permanencia de ACC durante OUTPUT y HALT.

Diapositiva 9 Demostración de la interfaz

Tiempo aproximado: 2 minutos.

1. Cambiar a VM Studio. Cargar el ejemplo Suma y mostrar que PC y ACC empiezan en cero.
2. Señalar Mem[10] = 10 y Mem[11] = 20. Explicar que ya están inicializados mediante DATA.
3. Pulsar Paso una vez. Señalar IR = LOAD 10, PC = 1 y ACC = 10.
4. Pulsar Paso otra vez. Señalar ACC = 30 y PC = 2.
5. Pulsar Paso para STORE. Seleccionar o localizar la celda 20 y mostrar el valor 30.
6. Completar OUTPUT y HALT. Mostrar la impresora virtual y el estado de finalización.
7. Reiniciar y ejecutar automáticamente para demostrar que ambos modos llegan al mismo resultado.
8. Volver a las diapositivas, sin dejar un diálogo abierto.

Frase de cierre de la demostración: “No solo observamos treinta como resultado; también conservamos el recorrido que explica cómo llegamos a él.”

Diapositiva 10 Saltos y bucles

Tiempo aproximado: 55 segundos.

“Los saltos muestran por qué PC no siempre avanza de forma lineal. En el caso condicional calculamos veinte menos veinte. Z se activa y JZ dirige la ejecución a la dirección seis, donde se carga ciento once. La rama que imprimiría novecientos noventa y nueve no se ejecuta. En el caso del bucle usamos JNZ para repetir mientras el contador no sea cero. El programa suma tres más dos más uno y termina con seis.”

“Que exista HALT en el archivo no significa que se vaya a alcanzar. Un salto podría repetir para siempre; por eso hay un límite de ciclos.”

Diapositiva 11 Dispositivos virtuales

Tiempo aproximado: 50 segundos.

“El controlador de teclado entrega enteros a INPUT. El controlador de disco lee los archivos ASM. El controlador de impresora recibe el valor de ACC al ejecutar OUTPUT. Cada uno tiene su adaptador separado. En el caso de entrada y salida, introducir siete deja ACC y memoria veinte en siete y emite siete. Si se cancela el diálogo, la instrucción queda pendiente y se permite reintentar sin contar un ciclo incompleto.”

Transición: “Mi compañero mostrará ahora cómo controlamos los errores y cómo conservamos las evidencias de estas ejecuciones.”

Guion del Integrante 3

Diapositiva 12 Manejo de errores

Tiempo aproximado: 1 minuto y 15 segundos.

“Distinguimos errores de carga y errores de ejecución. Una instrucción desconocida, una dirección fuera del rango o un programa sin HALT se rechazan al cargar. Una carga inválida conserva el programa anterior. Un error de ejecución ocurre cuando un programa válido intenta una operación incompatible, como leer una instrucción como si fuera un dato. En ese caso se conserva el PC fallido, se registra la causa y se exige reiniciar o cargar otro programa.”

“Por ejemplo, MULT diez es una instrucción inexistente. En cambio, LOAD cero seguido de HALT tiene sintaxis válida, pero falla al ejecutar porque la celda cero contiene el propio código. La aplicación informa ERROR, no una finalización correcta.”

Si hay tiempo, mostrar el error controlado en la GUI durante 20 segundos. No escribir programas nuevos largos frente al público.

Diapositiva 13 Reportes de ejecución

Tiempo aproximado: 1 minuto y 15 segundos.

“Los reportes conservan los resultados de la ejecución actual. El formato HTML se puede abrir sin Internet y preparar para impresión. CSV permite revisar los pasos en Excel. JSON conserva la evidencia estructurada, incluyendo programa, datos iniciales, entradas, registros, memoria, salida y fases. La evidencia también puede registrar una ejecución pausada o con error; no debe presentarse como aprobada simplemente porque fue exportada.”

“Para reproducir una prueba es importante conocer tanto el programa como sus condiciones iniciales. Un número final sin esas condiciones no explica el comportamiento de la máquina.”

Diapositiva 14 Pruebas y resultados

Tiempo aproximado: 1 minuto y 20 segundos.

“Diseñamos seis programas funcionales: suma, resta, almacenamiento, condicional, bucle y entrada y salida. Los valores esperados son treinta, veinte, treinta, ciento once, seis y siete respectivamente. Además comprobamos errores de carga y regresiones del motor. Las pruebas de interfaz verifican acciones como cargar, ejecutar, pausar, reiniciar y exportar.”

“El resultado que mostramos corresponde a una ejecución real del verificador. Las pruebas aumentan la confianza y permiten detectar cambios que rompan un comportamiento anterior. No demuestran que sea imposible encontrar un fallo en cualquier condición; por eso también ensayamos el recorrido que se utilizará durante la defensa.”

El conteo de regresiones debe leerse del informe generado `docs/EVIDENCIAS_PRUEBAS.md` para evitar memorizar una cifra que cambie después de una actualización.

Diapositiva 15 Organización del código

Tiempo aproximado: 1 minuto.

“La organización separa responsabilidades. motor.py contiene memoria, instrucciones, registros y ejecución. interfaz.py presenta los controles gráficos. consola.py permite el uso por terminal y la ejecución por archivo. reportes.py transforma el estado en evidencia y escribe los formatos de salida. Los ejemplos y las pruebas están en carpetas distintas del código de la aplicación. La documentación y las diapositivas acompañan la entrega.”

“Esta separación permite cambiar la presentación o añadir un reporte sin duplicar la lógica de las instrucciones. Las interfaces consultan el mismo estado del motor.”

Diapositiva 16 Conclusiones y límites

Tiempo aproximado: 1 minuto y 10 segundos.

“El resultado es una herramienta didáctica que hace visible el ciclo de ejecución y permite contrastar los cambios con programas pequeños. La suma demuestra la relación entre instrucciones, registros y memoria; los saltos demuestran el control del flujo; las evidencias conservan el resultado de cada prueba. El modelo

tiene límites deliberados: cien celdas, diez instrucciones y una ejecución secuencial. No representa un sistema operativo completo ni simula concurrencia, interrupciones o una CPU física.”

“Como ampliaciones futuras podrían incorporarse etiquetas simbólicas o nuevas instrucciones, acompañadas de documentación y pruebas. Con esto concluimos la demostración y quedamos atentos a sus preguntas.”

Programas preparados para la demostración

Suma

```
DATA 10 10
DATA 11 20
LOAD 10
ADD 11
STORE 20
OUTPUT
HALT
```

Resultado: ACC = 30, Mem[20] = 30, impresora = [30], PC = 5, IR = HALT.

Error de carga

```
MULT 10
HALT
```

Resultado: carga rechazada por instrucción inexistente. El programa anterior permanece disponible.

Error durante la ejecución

```
LOAD 0
HALT
```

Resultado: ERROR al leer código como dato, PC conservado en 0. Reiniciar o cargar un programa válido antes de continuar.

Bucle que no alcanza HALT

```
JMP 0
HALT
```

Elegir un límite pequeño antes de ejecutar. Resultado: interrupción por límite de ciclos, no terminación normal. Cargar nuevamente el ejemplo Suma para salir del caso de error.

Preguntas frecuentes de defensa

Pregunta	Respuesta recomendada
¿Qué es una máquina virtual en este proyecto?	Una representación por software de una arquitectura didáctica con instrucciones propias. No es un virtualizador de sistemas operativos.
¿Por qué PC y ACC no son lo mismo?	PC guarda una dirección de instrucción y ACC guarda un dato utilizado en operaciones.
¿Qué conserva IR después de un paso?	La última instrucción obtenida de memoria, hasta que el siguiente Fetch lo reemplace.
¿Cuándo aumenta PC?	Después de Decode y antes de Execute; un salto puede reemplazarlo durante Execute.
¿Por qué el PC final de la suma vale cinco?	Se completan las instrucciones en direcciones cero a cuatro; HALT también prepara la continuación secuencial.
¿Qué diferencia hay entre LOAD 10 y DATA 10 10?	LOAD lee una celda durante la ejecución; DATA inicializa esa celda durante la carga y no consume ciclos.
¿Se puede escribir LOAD ACC?	No. ACC es implícito y el operando de LOAD debe ser una dirección entera.
¿Cuándo se activan Z y N?	Z si ACC es cero y N si ACC es negativo, después de cada instrucción completada.
¿JZ cambia el acumulador?	No; consulta Z y puede cambiar el destino de PC.

Pregunta	Respuesta recomendada
¿Por qué exigir HALT si puede existir un bucle infinito?	HALT define una terminación explícita del lenguaje, pero la ejecución puede no alcanzarlo. El límite de ciclos protege ese caso.
¿Qué pasa si STORE sobrescribe una instrucción?	La escritura usa memoria unificada; si PC llega después a esa celda convertida en dato, Fetch detecta que no hay una instrucción válida.
¿Una carga inválida borra lo anterior?	No. Se valida antes de sustituir el estado del programa activo.
¿Cancelar INPUT cuenta como un ciclo?	No. La instrucción queda pendiente y se permite reintentar.
¿Por qué utilizar controladores?	Separan la lógica del motor de la entrada, los archivos y la salida, y permiten probar con funciones simuladas.
¿Por qué no basta con mostrar un resultado correcto?	También debe explicarse el recorrido: programa, entradas, estados intermedios y memoria modificada.
¿Las pruebas garantizan ausencia total de errores?	No. Verifican condiciones concretas y regresiones conocidas. Se complementan con revisión y ensayo de la demostración.
¿Por qué el proyecto está en Python?	La propuesta lista Python al inicio y también contiene referencias a Java. El grupo debe presentar la autorización real que tenga para la elección de Python.
¿Qué queda fuera del alcance?	Concurrencia, interrupciones, caché, protección por procesos y ancho de palabra fijo.

Plan de contingencia

Si no abre la interfaz, comprobar primero `python -m tkinter`. Si el equipo no tiene Tkinter, utilizar `python simulador_vm.py --cli` y explicar los mismos registros. No instalar paquetes durante la exposición.

Si el proyector recorta la ventana, reducir su tamaño y usar las pestañas; mostrar la captura incluida en la diapositiva para orientar al público. Si la demostración tarda demasiado, ejecutar Suma automáticamente y explicar la tabla de la diapositiva 8.

Si aparece un error esperado, identificar si ocurrió al cargar o al ejecutar. Explicar el diagnóstico y recuperar con Reiniciar o cargando Suma. No presentar un ERROR como si fuera un resultado correcto.

Si el archivo de presentación no abre, utilizar la copia PDF o las imágenes exportadas de las diapositivas que acompañen el material. Los reportes y programas de ejemplo deben estar guardados localmente.

Ensayo final

Cada integrante debe ejecutar la suma sin ayuda y explicar un salto. El Integrante 1 debe poder distinguir memoria, PC e IR; el Integrante 2 debe justificar cada cambio de ACC; el Integrante 3 debe explicar el reporte y el límite de ciclos. Ensayar las dos transiciones entre compañeros y dejar un minuto de margen antes del cierre.